📘 Día 21

Clases y objetos

Python es un lenguaje orientado a objetos en su núcleo. Esto significa que absolutamente todo lo que existe en tiempo de ejecución — un número, un string, una lista, una función — es un objeto en el heap, instancia de alguna clase. Una clase es el molde que define qué datos tiene un objeto (atributos) y qué puede hacer (métodos). El objeto es la instancia concreta creada a partir de ese molde.

Desde el primer día de este desafío hemos usado clases sin saberlo: cada vez que escribimos `'texto'` o `[1,2,3]` estamos creando instancias de `str` y `list` respectivamente.

python

```
# Verificando que todo en Python es un objeto de alguna clase

num = 10

type(num)      # <class 'int'>    → num es instancia de la clase int


string = 'string'

type(string)   # <class 'str'>    → string es instancia de la clase str


boolean = True

type(boolean)  # <class 'bool'>   → boolean es instancia de la clase bool


lst = []

type(lst)      # <class 'list'>   → lst es instancia de la clase list


tpl = ()

type(tpl)      # <class 'tuple'>  → tpl es instancia de la clase tuple
```

```
set1 = set()

type(set1)     # <class 'set'>   → set1 es instancia de la clase set


dct = {}

type(dct)      # <class 'dict'>   → dct es instancia de la clase dict
```

Crear una clase

Para definir una clase se usa la palabra clave `class` seguida del nombre en **CamelCase** y dos puntos. El cuerpo de la clase contiene sus atributos y métodos.

```
# syntax
class NombreClase:
# cuerpo de la clase
```

Ejemplo:

```
python
class Person:   # define la clase Person — crea un objeto PyTypeObject en el heap
    pass        # pass indica que el cuerpo está vacío por ahora; clase válida pero sin
comportamiento

print(Person)   # imprime la representación de la clase, no de una instancia
# → <class '__main__.Person'>
```

Crear un objeto

Instanciar una clase significa llamarla como si fuera una función. Esto reserva memoria en el heap para el nuevo objeto y retorna una referencia a él.

Sintaxis:

```
python
variable = NombreClase()
python
class Person:
    pass

p = Person()    # llama a Person() → crea instancia en el heap → p apunta a ella en
globals()
print(p)        # imprime la dirección del objeto en memoria
# → <__main__.Person object at 0x10804e510>
```

Python 3.11
known limitations

```
1 class Person:
2   pass
3 print(Person)
4
5 p = Person()
6 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 4

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
Person → Person class

Python 3.11
known limitations

```
1 class Person:
2   pass
3 print(Person)
4
5 p = Person()
6 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 4

Print output (drag lower right corner to resize)

```
<class '__main__.Person'>
```

Frames | Objects

Global frame
Person → Person class

Python 3.11
known limitations

```
1 class Person:
2   pass
3 print(Person)
4
5 p = Person()
6 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

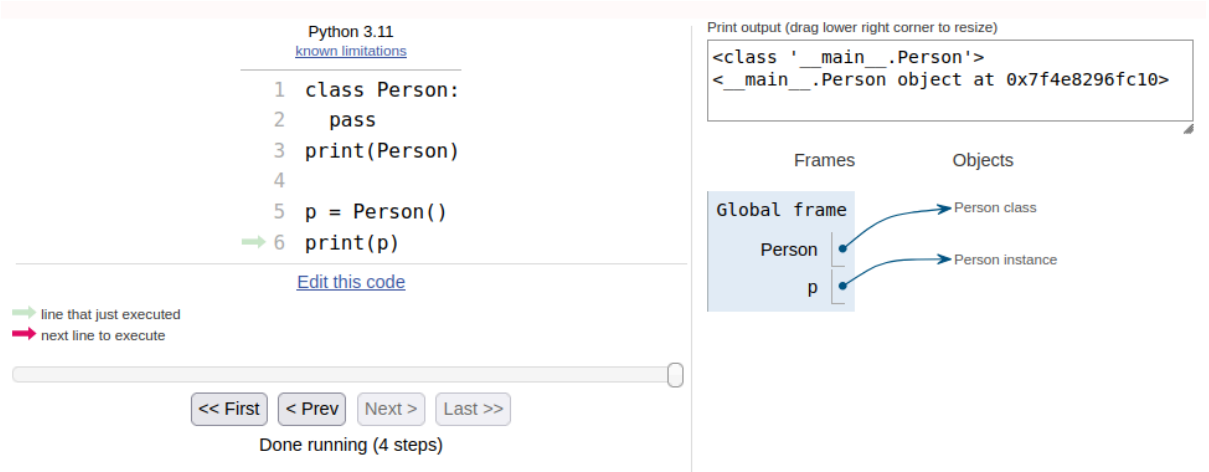
Step 4 of 4

Print output (drag lower right corner to resize)

```
<class '__main__.Person'>
```

Frames | Objects

Global frame
Person → Person class
p → Person instance



PythonTutor — Pasos 1 a 4

Paso 2 de 4 (next line: `print(Person)`): PythonTutor muestra en el Global Frame una entrada `Person` que apunta a un objeto "Person class" en el panel Objects. Lo que ocurre: al procesar `class Person: pass`, el intérprete crea un objeto `PyTypeObject` en el heap y registra la clave `"Person"` en `globals()` apuntando a él. El panel Objects muestra la clase como un recuadro separado — esto es correcto, una clase es un objeto de primera clase en Python.

Omisión importante: PythonTutor no muestra que el objeto clase también tiene su propio namespace interno (un dict) donde vivirán sus métodos. Tampoco muestra que `Person` hereda automáticamente de `object`, la clase base de todo en Python.

Paso 3 de 4 (ejecutando `print(Person)`): El output muestra `<class '__main__.Person'>`. El prefijo `__main__` indica el módulo donde fue definida la clase — en este caso el script principal. PythonTutor no muestra la búsqueda LEGB de `print` ni la resolución del argumento `Person`.

Paso 4 de 4 (next line: `print(p)`): Aparece en el Global Frame una nueva entrada `p` que apunta a un recuadro "Person instance" en Objects. La instancia aparece vacía porque la clase no tiene atributos todavía. **Omisión:** el objeto instancia ya existe en el heap con su `ob_refcnt`, `ob_type` apuntando a la clase Person, y un `__dict__` vacío para sus atributos futuros — nada de esto es visible en el diagrama.

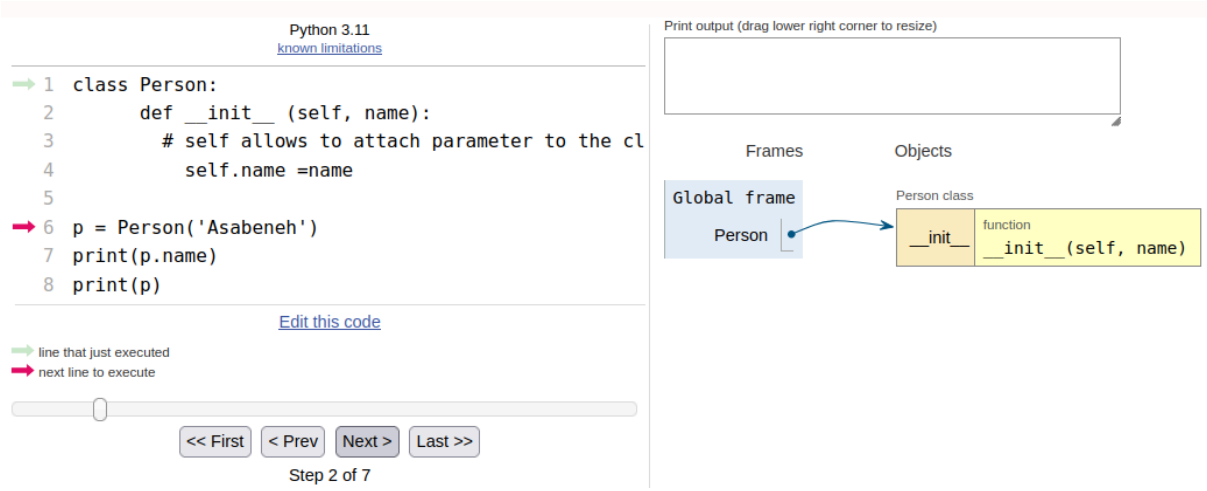
Constructor de clase

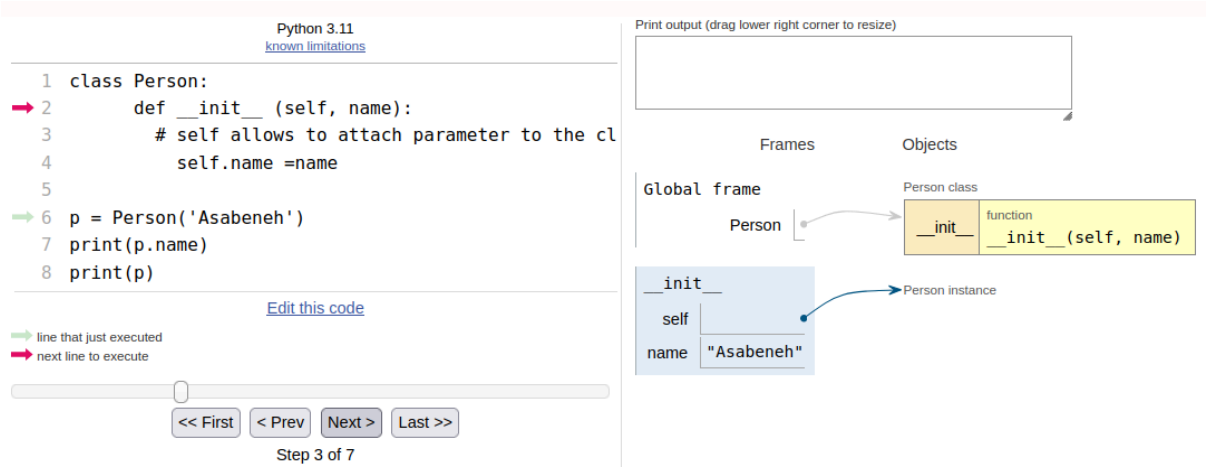
Una clase sin constructor no puede recibir datos al momento de ser instanciada. `__init__` es el método especial (dunder) que Python llama automáticamente justo después de crear el objeto, permitiendo inicializar sus atributos. El parámetro `self` es la referencia a la instancia recién creada.

Sintaxis:

python
```
class NombreClase:
    def __init__(self, param1, param2):
        self.param1 = param1    # vincula el argumento como atributo del objeto
        self.param2 = param2
```
python
```
class Person:
    def __init__(self, name):      # __init__ recibe self (la instancia) y name (argumento)
        # self es un puntero a la instancia en el heap — no es una copia, es la referencia real
        self.name = name          # crea el atributo name en el __dict__ de la instancia

p = Person('Asabeneh')  # 1) crea objeto en heap, 2) llama __init__(p, 'Asabeneh') automáticamente
print(p.name)          # busca 'name' en el __dict__ de la instancia apuntada por p
# → Asabeneh
print(p)               # sin __str__ definido, usa la representación default heredada de object
# → <__main__.Person object at 0x2abf46907e80>
```





Python 3.11
known limitations

```
1 class Person:
2       def __init__ (self, name):
3         # self allows to attach parameter to the cl
4           self.name =name
5
6 p = Person('Asabeneh')
7 print(p.name)
8 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 5 of 7

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
Person

Person class
__init__ | function __init__(self, name)

__init__
self
name "Asabeneh"
Return value None

Person instance
name "Asabeneh"

Python 3.11
known limitations

```
1 class Person:
2       def __init__ (self, name):
3         # self allows to attach parameter to the cl
4           self.name =name
5
6 p = Person('Asabeneh')
7 print(p.name)
8 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 6 of 7

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
Person
p

Person class
__init__ | function __init__(self, name)

Person instance
name "Asabeneh"

Python 3.11
known limitations

```
1 class Person:
2       def __init__ (self, name):
3         # self allows to attach parameter to the cl
4           self.name =name
5
6 p = Person('Asabeneh')
7 print(p.name)
8 print(p)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 7 of 7

Print output (drag lower right corner to resize)

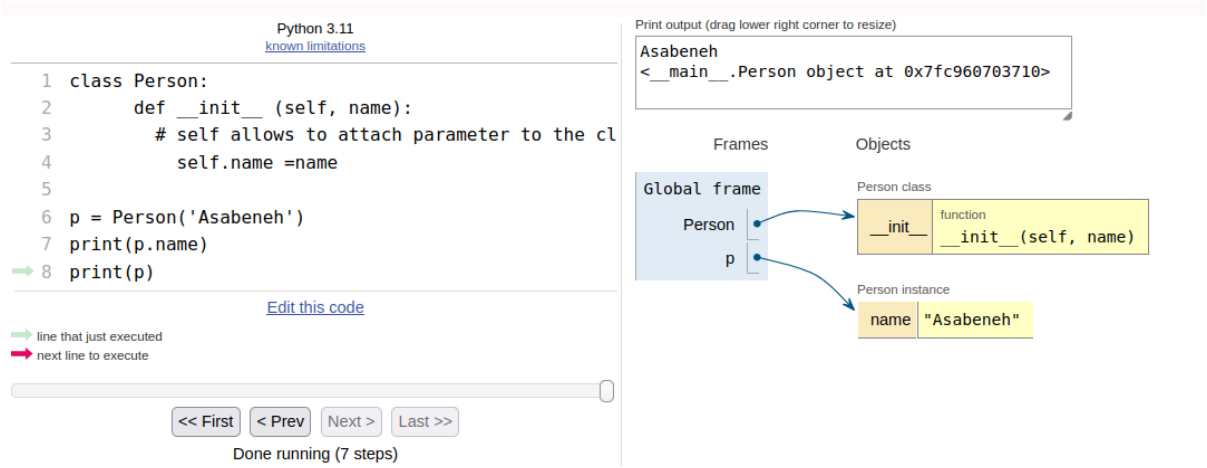
```
Asabeneh
```

Frames | Objects

Global frame
Person
p

Person class
__init__ | function __init__(self, name)

Person instance
name "Asabeneh"



PythonTutor — Pasos del constructor simple

Paso 2 de 7 (registrando la clase): Aparece Person class en Objects con `__init__` listado como función. Esto confirma que los métodos son objetos función almacenados en el namespace de la clase, no en las instancias. **Omisión:** cada método es un objeto `PyFunctionObject` en el heap; PythonTutor lo simplifica como una etiqueta.

Paso 3 de 7 (ejecutando `p = Person('Asabeneh')`, entrando a `__init__`): Aparece un nuevo frame `__init__` debajo del Global Frame con las variables locales `self` y `name`. `self` apunta ya a la "Person instance" en Objects, `name` contiene `"Asabeneh"`. Esto muestra el momento exacto en que `__init__` está ejecutándose — es un frame de función como cualquier otro en el call stack.

Omisión crítica: Python primero llama a `__new__` (que reserva la memoria y crea el objeto vacío) y luego llama a `__init__` (que lo inicializa). PythonTutor omite completamente `__new__`, dando la impresión de que `__init__` crea el objeto, cuando en realidad solo lo configura.

Paso 4 de 7 (ejecutando `self.name = name`): En el panel Objects, la Person instance ahora muestra el atributo `name: "Asabeneh"`. Esto es la escritura en el `__dict__` de la instancia. `self.name = name` es equivalente a `self.__dict__['name'] = name` — inserta una clave en el diccionario de atributos del objeto.

Paso 5 de 7 (return de `__init__`): El frame `__init__` muestra `Return value: None`. `__init__` siempre retorna `None` explícitamente — si retornara otra cosa, Python lanzaría `TypeError`. El frame desaparece del call stack.

Pasos 6 y 7 (ejecutando `print(p.name)` y `print(p)`): `p.name` se resuelve buscando `'name'` primero en el `__dict__` de la instancia, luego en la clase — lo encuentra en la instancia inmediatamente. `print(p)` sin `__str__` delega a `__repr__` heredado de `object`, que produce la dirección de memoria.

Constructor con múltiples parámetros

Añadamos más parámetros a la función constructor.

python
```
class Person:
    def __init__(self, firstname, lastname, age, country, city):
        # cada self.X crea una entrada en el __dict__ de la instancia
        self.firstname = firstname  # atributo firstname vinculado al argumento
        self.lastname = lastname    # atributo lastname
        self.age = age              # atributo age
        self.country = country      # atributo country
        self.city = city            # atributo city

p = Person('Asabeneh', 'Yetayeh', 250, 'Finland', 'Helsinki')
# llama __init__ con los 5 argumentos posicionales en orden

print(p.firstname)  # busca en p.__dict__ → 'Asabeneh'
print(p.lastname)   # → 'Yetayeh'
print(p.age)        # → 250
print(p.country)    # → 'Finland'
print(p.city)       # → 'Helsinki'
```

Métodos de Objetos

Los métodos son funciones definidas dentro de la clase que operan sobre la instancia. Siempre reciben `self` como primer parámetro — que es la referencia al objeto sobre el que se llama el método.

Sintaxis:

python
```
class NombreClase:
    def nombre_metodo(self, param):
        return ...          # puede retornar un valor o modificar self
```
python
```
class Person:
    def __init__(self, firstname, lastname, age, country, city):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.country = country
        self.city = city
```

```
    def person_info(self):  # método de instancia: recibe self automáticamente al llamarlo
        # f-string que accede a los atributos de la instancia a través de self
        return f'{self.firstname} {self.lastname} is {self.age} years old. He lives in {self.city},
{self.country}'

p = Person('Asabeneh', 'Yetayeh', 250, 'Finland', 'Helsinki')
print(p.person_info())  # Python traduce p.person_info() a Person.person_info(p)
# → Asabeneh Yetayeh is 250 years old. He lives in Helsinki, Finland
```

Métodos predeterminados de objetos

Los parámetros del constructor pueden tener valores por defecto, lo que permite instanciar la clase sin pasar argumentos. Si se pasan argumentos, sobreescriben los valores por defecto.

Sintaxis:

python
```
def __init__(self, param='valor_default'):
    self.param = param
```
python
```
class Person:
    def __init__(self, firstname='Asabeneh', lastname='Yetayeh',
                 age=250, country='Finland', city='Helsinki'):
        # si no se pasa argumento, se usa el valor default declarado
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.country = country
        self.city = city

    def person_info(self):
        return f'{self.firstname} {self.lastname} is {self.age} years old. He lives in {self.city},
{self.country}.'

p1 = Person()   # usa todos los valores default — instancia válida sin argumentos
print(p1.person_info())
# → Asabeneh Yetayeh is 250 years old. He lives in Helsinki, Finland.

p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman city')  # sobreescribe todos los defaults
print(p2.person_info())
# → John Doe is 30 years old. He lives in Noman city, Nomanland.
```

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 25

Print output (drag lower right corner to resize)

Frames

Global frame
Person

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 25

Print output (drag lower right corner to resize)

Frames

Global frame
Person

__init__

self	
firstname	"Asabeneh"
lastname	"Yetayeh"
age	250
country	"Finland"
city	"Helsinki"

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Person instance

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 9 of 25

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
Person

__init__

self	
firstname	"Asabeneh"
lastname	"Yetayeh"
age	250
country	"Finland"
city	"Helsinki"
Return value	None

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Person instance

age	250
city	"Helsinki"
country	"Finland"
firstname	"Asabeneh"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 10 of 25

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
Person
p1

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Person instance

age	250
city	"Helsinki"
country	"Finland"
firstname	"Asabeneh"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 11 of 25

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
Person
p1

person_info
self

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Person instance

age	250
city	"Helsinki"
country	"Finland"
firstname	"Asabeneh"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 p1 = Person()
13 print(p1.person_info())
14 p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman
15 print(p2.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 13 of 25

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
Person
p1

person_info
self
Return value "Asabeneh Yetayeh is 250 years old. He lives in Helsinki, Finland."

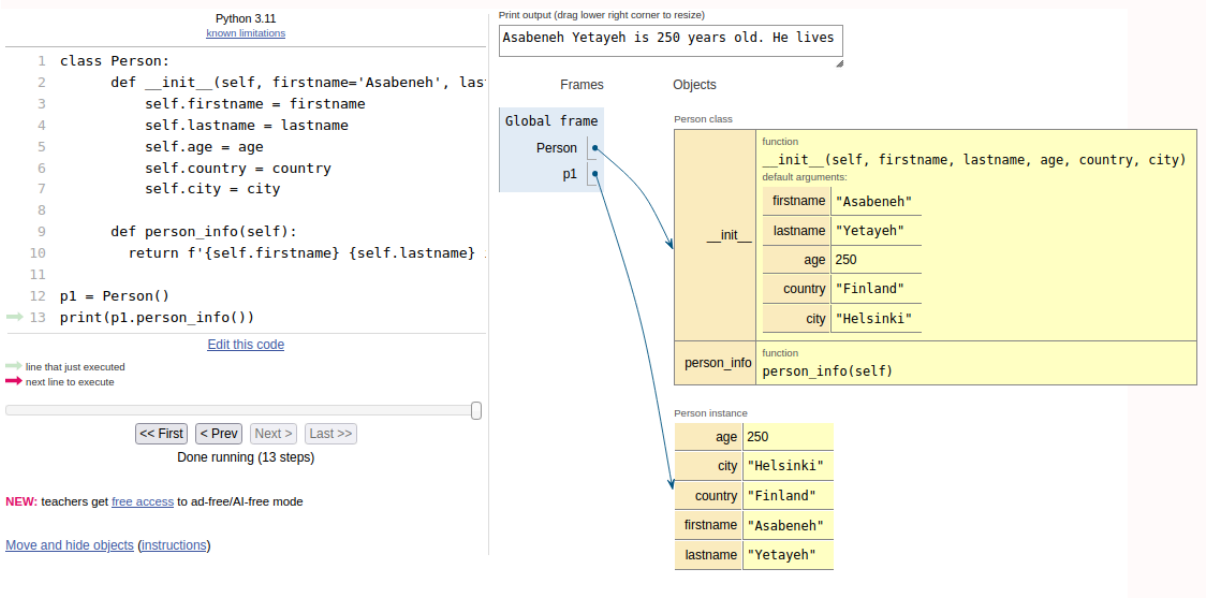
Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Person instance

age	250
city	"Helsinki"
country	"Finland"
firstname	"Asabeneh"
lastname	"Yetayeh"



PythonTutor — Pasos con defaults (Paso 2 de 25)

El panel Objects muestra la Person class con `__init__` listado incluyendo sus **default arguments**: `firstname="Asabeneh"`, `lastname="Yetayeh"`, `age=250`, `country="Finland"`, `city="Helsinki"`. También aparece `person_info` como función. **Omisión:** los valores default no se almacenan en el frame de `__init__` sino en el objeto función mismo, en su atributo `__defaults__` — una tupla. PythonTutor los muestra dentro de la clase como si fueran atributos de clase, lo cual es una simplificación.

Paso 3 de 25 (entrando a `__init__` desde `p1 = Person()`): Aparece el frame `__init__` con `self` apuntando a la nueva instancia vacía. Los parámetros `firstname`, `lastname`, etc. toman sus valores default. **Omisión:** Python copió los valores default desde `__defaults__` del objeto función al frame local — esto no es visible.

Paso 9 de 25 (fin de `__init__`, instancia completa): La Person instance en Objects muestra todos los atributos: `age=250`, `city="Helsinki"`, `country="Finland"`, `firstname="Asabeneh"`, `lastname="Yetayeh"`. Los atributos aparecen en orden alfabético — esto es un detalle de la visualización de PythonTutor, no el orden real de inserción en `__dict__`.

Método para modificar los valores predeterminados de clase

Un atributo puede ser una lista. Se inicializa vacía en el constructor y se modifica mediante métodos. Como las listas son mutables, `append` modifica el objeto lista en el heap sin crear uno nuevo.

python
```
class Person:
```

```python
    def __init__(self, firstname='Asabeneh', lastname='Yetayeh',
                 age=250, country='Finland', city='Helsinki'):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.country = country
        self.city = city
        self.skills = []          # cada instancia obtiene su PROPIA lista vacía en el heap

    def person_info(self):
        return f'{self.firstname} {self.lastname} is {self.age} years old. He lives in {self.city},
{self.country}.'

    def add_skill(self, skill):     # método que modifica el atributo skills de la instancia
        self.skills.append(skill)   # append muta la lista existente, no crea una nueva

p1 = Person()
print(p1.person_info())
# → Asabeneh Yetayeh is 250 years old. He lives in Helsinki, Finland.

p1.add_skill('HTML')       # agrega 'HTML' a p1.skills
p1.add_skill('CSS')        # agrega 'CSS'
p1.add_skill('JavaScript')  # agrega 'JavaScript'

p2 = Person('John', 'Doe', 30, 'Nomanland', 'Noman city')
print(p2.person_info())
# → John Doe is 30 years old. He lives in Noman city, Nomanland.

print(p1.skills)  # → ['HTML', 'CSS', 'JavaScript']
print(p2.skills)  # → []  ← p2 tiene su propia lista, independiente de p1
```

> `self.skills = []` en `__init__` garantiza que cada instancia tenga su propia lista. Si se definiera `skills = []` como atributo de clase (fuera de `__init__`), todas las instancias compartirían la misma lista — un error frecuente.

Herencia

La herencia permite crear una clase nueva (hija) que reutiliza todos los atributos y métodos de una clase existente (padre), pudiendo agregar o modificar comportamiento. La clase hija no necesita reimplementar lo que ya existe en el padre.

Sintaxis:

python
```python
class ClaseHija(ClasePadre):
```

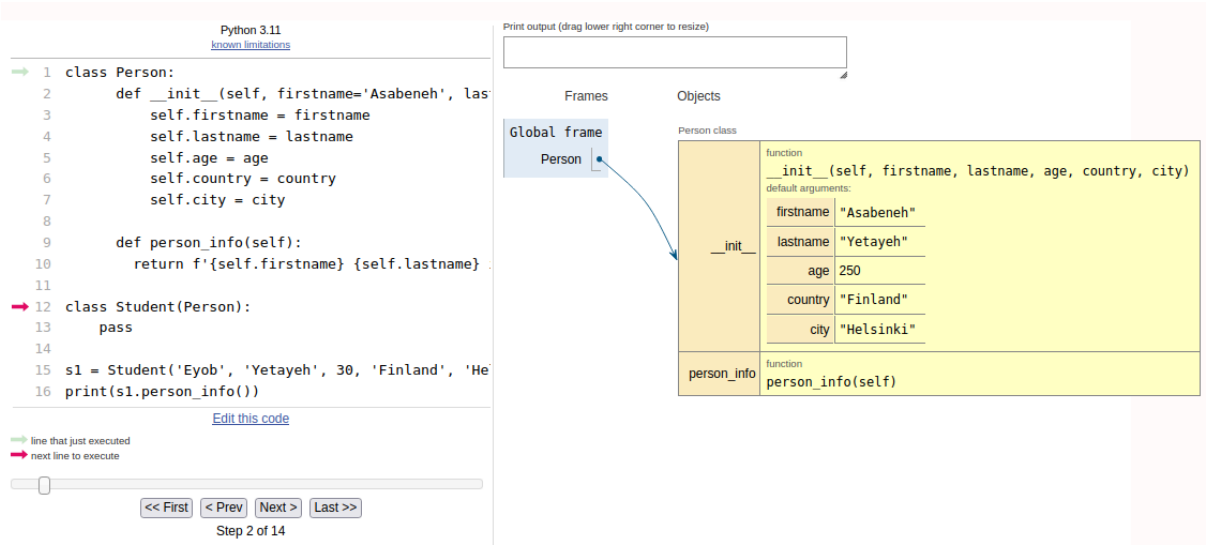
```
    pass    # hereda todo sin agregar nada
python
class Student(Person):  # Student hereda de Person — tiene acceso a __init__, person_info, add_skill
    pass            # no agrega ni sobreescribe nada; hereda el comportamiento completo

s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'Helsinki')
# llama al __init__ de Person porque Student no tiene el suyo propio
s2 = Student('Lidiya', 'Teklemariam', 28, 'Finland', 'Espoo')

print(s1.person_info())     # método heredado de Person, ejecutado sobre instancia Student
s1.add_skill('JavaScript')  # método heredado de Person
s1.add_skill('React')
s1.add_skill('Python')
print(s1.skills)            # → ['JavaScript', 'React', 'Python']

print(s2.person_info())
s2.add_skill('Organizing')
s2.add_skill('Marketing')
s2.add_skill('Digital Marketing')
print(s2.skills)            # → ['Organizing', 'Marketing', 'Digital Marketing']

# output
# Eyob Yetayeh is 30 years old. He lives in Helsinki, Finland.
# ['JavaScript', 'React', 'Python']
# Lidiya Teklemariam is 28 years old. He lives in Espoo, Finland.
# ['Organizing', 'Marketing', 'Digital Marketing']
```



Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 14

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- Person
- Student

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 4 of 14

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- Person
- Student

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Student instance

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 10 of 14

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- Person
- Student

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
Return value	None

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 11 of 14

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
- Person
- Student
- s1

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 12 of 14

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
- Person
- Student
- s1

person_info
- self

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
lastname	"Yetayeh"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8
9      def person_info(self):
10         return f'{self.firstname} {self.lastname}
11
12 class Student(Person):
13     pass
14
15 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
16 print(s1.person_info())
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 13 of 14

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

Frames

Global frame
- Person
- Student
- s1

person_info
- self

Objects

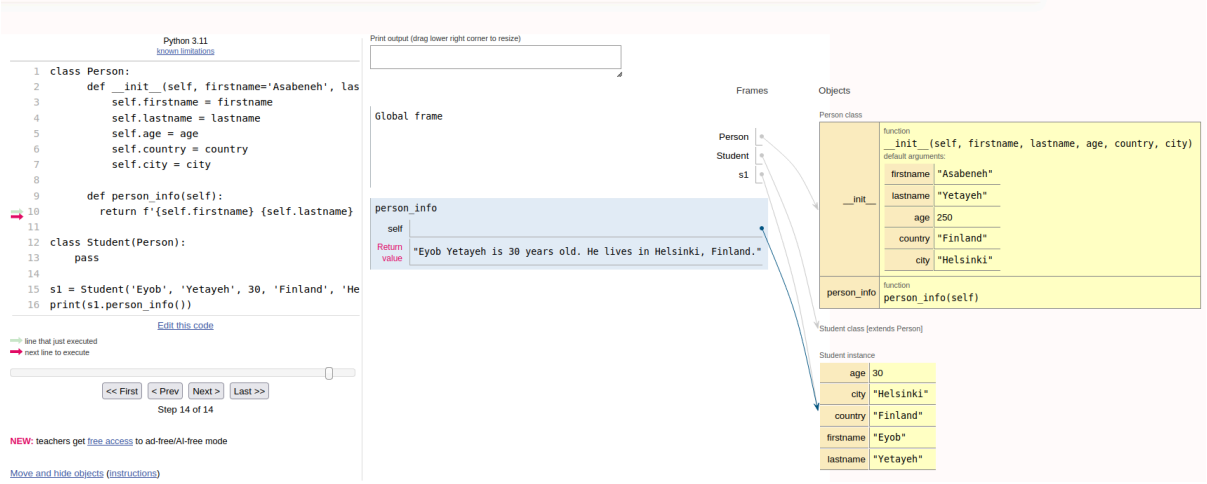
Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
person_info	function person_info(self)

Student class [extends Person]

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
lastname	"Yetayeh"



PythonTutor — Herencia simple (Pasos 2 a 14)

Paso 2 de 14 (registrando Person class): Aparece solo Person en el Global Frame apuntando a su clase. Student todavía no existe. La Person class muestra `__init__` y `person_info` como funciones.

Paso 3 de 14 (registrando Student class): Aparece Student en el Global Frame. En Objects aparece "Student class [extends Person]" — PythonTutor indica la relación de herencia con esa etiqueta. **Omisión:** Student tiene internamente un puntero `__bases__ = (Person,)` y una MRO (Method Resolution Order) que determina el orden de búsqueda de métodos: `Student → Person → object`. Esto no es visible pero explica por qué `s1.person_info()` funciona.

Paso 4 de 14 (entrando a `__init__` desde `s1 = Student(...)`): El frame `__init__` que aparece pertenece a Person, no a Student — porque Student no tiene el suyo. `self` apunta a una "Student instance". Esto demuestra que la búsqueda de `__init__` siguió la MRO hasta encontrarlo en Person. **Omisión:** la MRO es el mecanismo real; PythonTutor no lo muestra.

Paso 10 de 14 (instancia s1 creada, ejecutando `print(s1.person_info())`): s1 aparece en el Global Frame apuntando a la Student instance con todos sus atributos. El frame `person_info` aparece con `self` apuntando a la misma instancia. La búsqueda de `person_info` en `s1` fue: no existe en Student instance → no existe en Student class → encontrado en Person class.

No llamamos al constructor **init**() en la clase infantil. Si no lo llamamos, entonces todavía podemos acceder a todas las propiedades del padre. Pero si llamamos al constructor podemos acceder a las propiedades principales llamando *al super*.
Podemos agregar un nuevo método al niño o podemos anular los métodos de la clase de padres creando el mismo nombre de método en la clase infantil. Cuando agregamos la función **init**(), la clase infantil ya no heredará la función **init**() del padre.

Sobreescritura del método padre (Override)

Cuando la clase hija define un método con el mismo nombre que el padre, reemplaza al del padre para esa clase. Se usa `super().__init__()` para llamar al constructor del padre y no tener que repetir su código.

Sintaxis:

python

```
class ClaseHija(ClasePadre):

    def __init__(self, param_nuevo, *params_padre):

        self.param_nuevo = param_nuevo          # atributo propio de la hija

        super().__init__(*params_padre)         # delega el resto al padre


    def metodo_sobreescrito(self):              # reemplaza al método del padre

        ...
```

python

```
class Student(Person):

    def __init__(self, firstname='Asabeneh', lastname='Yetayeh',

                 age=250, country='Finland', city='Helsinki', gender='male'):

        self.gender = gender        # atributo exclusivo de Student, se asigna ANTES de super()

        super().__init__(firstname, lastname, age, country, city)

        # super().__init__ llama al __init__ de Person, que asigna los 5 atributos restantes

        # evita duplicar el código de Person


    def person_info(self):          # sobreescribe person_info de Person

        gender = 'He' if self.gender == 'male' else 'She'   # lógica nueva en la hija

        return f'{self.firstname} {self.lastname} is {self.age} years old. {gender} lives in {self.city},
{self.country}.'


s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'Helsinki', 'male')
```

```python
s2 = Student('Lidiya', 'Teklemariam', 28, 'Finland', 'Espoo', 'female')


print(s1.person_info())     # usa la versión de Student, no la de Person
s1.add_skill('JavaScript')  # add_skill sigue heredado de Person sin cambios
s1.add_skill('React')
s1.add_skill('Python')
print(s1.skills)


print(s2.person_info())     # gender='female' → pronombre 'She'
s2.add_skill('Organizing')
s2.add_skill('Marketing')
s2.add_skill('Digital Marketing')
print(s2.skills)


# Eyob Yetayeh is 30 years old. He lives in Helsinki, Finland.
# ['JavaScript', 'React', 'Python']
# Lidiya Teklemariam is 28 years old. She lives in Espoo, Finland.
# ['Organizing', 'Marketing', 'Digital Marketing']
```

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8          self.skills = []
9
10     def person_info(self):
11       return f'{self.firstname} {self.lastname}
12     def add_skill(self, skill):
13         self.skills.append(skill)
14
15 class Student(Person):
16     def __init__ (self, firstname='Asabeneh', last
17         self.gender = gender
18         super().__init__(firstname, lastname,age,
19     def person_info(self):
20         gender = 'He' if self.gender =='male' else
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 15

Frames

Global frame
Person

Objects

Person class

__init__	function `__init__(self, firstname, lastname, age, country, city)` default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
add_skill	function `add_skill(self, skill)`
person_info	function `person_info(self)`

Python 3.11
known limitations

```
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8          self.skills = []
9
10     def person_info(self):
11       return f'{self.firstname} {self.lastname}
12     def add_skill(self, skill):
13         self.skills.append(skill)
14
15 class Student(Person):
16     def __init__ (self, firstname='Asabeneh', last
17         self.gender = gender
18         super().__init__(firstname, lastname,age,
19     def person_info(self):
20         gender = 'He' if self.gender =='male' else
21         return f'{self.firstname} {self.lastname}
22
23 s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 15

NEW: teachers get free access to ad-free/AI-free mode

Frames

Global frame
Person
Student

Objects

Person class

__init__	function `__init__(self, firstname, lastname, age, country, city)` default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
add_skill	function `add_skill(self, skill)`
person_info	function `person_info(self)`

Student class [extends Person]

__init__	function `__init__(self, firstname, lastname, age, country, city, gender)` default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki" gender: "male"
person_info	function `person_info(self)`

Python 3.11
known limitations

```
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8          self.skills = []
9
10      def person_info(self):
11        return f'{self.firstname} {self.lastname}
12      def add_skill(self, skill):
13          self.skills.append(skill)
14
15  class Student(Person):
16      def __init__ (self, firstname='Asabeneh', last
17          self.gender = gender
18          super().__init__(firstname, lastname,age,
19      def person_info(self):
20          gender = 'He' if self.gender =='male' else
21          return f'{self.firstname} {self.lastname}
22
23  s1 = Student('Eyob', 'Yetayeh', 30, 'Finland', 'He
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 4 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Frames

Global frame
- Person
- Student

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
gender	"male"
__class__	

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
add_skill	function add_skill(self, skill)
person_info	function person_info(self)

Student class [extends Person]

__init__	function __init__(self, firstname, lastname, age, country, city, gender) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki" gender: "male"
person_info	function person_info(self)

Student instance

Python 3.11
known limitations

```
1   class Person:
2       def __init__(self, firstname='Asabeneh', las
3           self.firstname = firstname
4           self.lastname = lastname
5           self.age = age
6           self.country = country
7           self.city = city
8           self.skills = []
9
10      def person_info(self):
11        return f'{self.firstname} {self.lastname}
12      def add_skill(self, skill):
13          self.skills.append(skill)
14
15  class Student(Person):
16      def __init__ (self, firstname='Asabeneh', last
17          self.gender = gender
18          super().__init__(firstname, lastname,age,
19      def person_info(self):
20          gender = 'He' if self.gender =='male' else
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 7 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Frames

Global frame
- Person
- Student

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
gender	"male"
__class__	

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki"
add_skill	function add_skill(self, skill)
person_info	function person_info(self)

Student class [extends Person]

__init__	function __init__(self, firstname, lastname, age, country, city, gender) default arguments: firstname: "Asabeneh" lastname: "Yetayeh" age: 250 country: "Finland" city: "Helsinki" gender: "male"
person_info	function person_info(self)

Student instance

gender	"male"

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8          self.skills = []
9
10     def person_info(self):
11         return f'{self.firstname} {self.lastname}
12     def add_skill(self, skill):
13         self.skills.append(skill)
14
15 class Student(Person):
16     def __init__ (self, firstname='Asabeneh', lasti
17         self.gender = gender
18         super().__init__(firstname, lastname,age,
19     def person_info(self):
20         gender = 'He' if self.gender =='male' else
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 14 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Frames

Global frame

Person	
Student	

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
gender	"male"
__class__	

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
Return value	None

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
add_skill	function add_skill(self, skill)
person_info	function person_info(self)

Student class [extends Person]

__init__	function __init__(self, firstname, lastname, age, country, city, gender) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki" gender "male"
person_info	function person_info(self)

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
gender	"male"
lastname	"Yetayeh"
skills	

Empty list

Python 3.11
known limitations

```
1  class Person:
2      def __init__(self, firstname='Asabeneh', las
3          self.firstname = firstname
4          self.lastname = lastname
5          self.age = age
6          self.country = country
7          self.city = city
8          self.skills = []
9
10     def person_info(self):
11         return f'{self.firstname} {self.lastname}
12     def add_skill(self, skill):
13         self.skills.append(skill)
14
15 class Student(Person):
16     def __init__ (self, firstname='Asabeneh', lasti
17         self.gender = gender
18         super().__init__(firstname, lastname,age,
19     def person_info(self):
20         gender = 'He' if self.gender =='male' else
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 15 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Frames

Global frame

Person	
Student	

__init__

self	
firstname	"Eyob"
lastname	"Yetayeh"
age	30
country	"Finland"
city	"Helsinki"
gender	"male"
__class__	
Return value	None

Objects

Person class

__init__	function __init__(self, firstname, lastname, age, country, city) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki"
add_skill	function add_skill(self, skill)
person_info	function person_info(self)

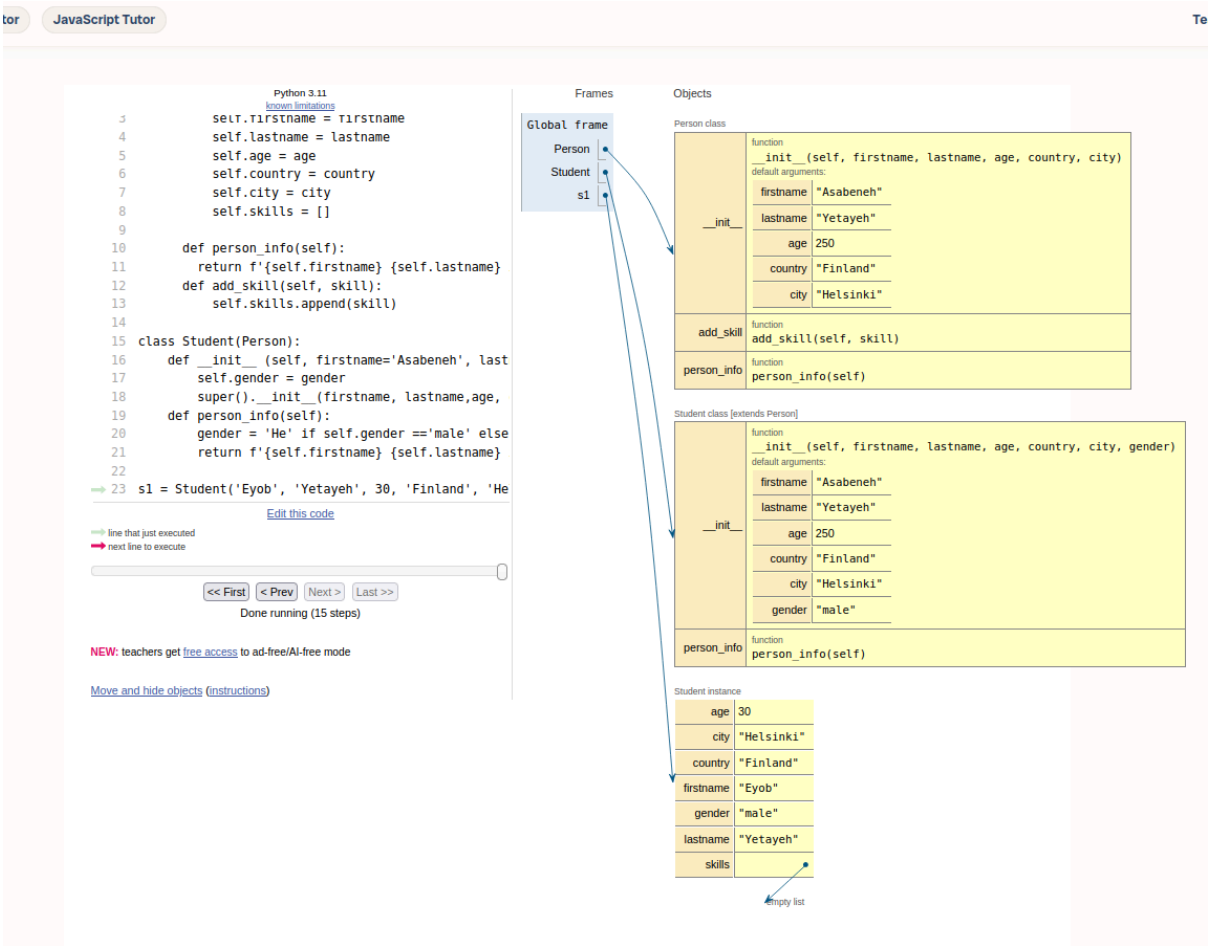
Student class [extends Person]

__init__	function __init__(self, firstname, lastname, age, country, city, gender) default arguments: firstname "Asabeneh" lastname "Yetayeh" age 250 country "Finland" city "Helsinki" gender "male"
person_info	function person_info(self)

Student instance

age	30
city	"Helsinki"
country	"Finland"
firstname	"Eyob"
gender	"male"
lastname	"Yetayeh"
skills	

Empty list



PythonTutor — Override con super() (Pasos 2 a 15)

Paso 2 de 15 (registrando Person class): Person class muestra `__init__`, `add_skill` y `person_info`. Ahora `add_skill` es visible porque fue agregado en la sección anterior. La Person class tiene los default arguments en `__init__`.

Paso 3 de 15 (registrando Student class): Student class [extends Person] aparece con su propio `__init__` (diferente al de Person) y su propio `person_info`. Ambas clases coexisten en Objects — cada una con su namespace propio. La relación de herencia existe pero los namespaces son separados.

Paso 4 de 15 (entrando al `__init__` de Student desde `s1 = Student(...)`): El frame `__init__` pertenece ahora a Student (no a Person como antes). Se ven los 6 parámetros incluyendo `gender='male'`. `self` apunta a la Student instance. La primera línea ejecutada es `self.gender = gender`.

Paso 7 de 15 (ejecutando `super().__init__(...)`): Aparece un segundo frame `__init__` apilado — este es el de Person. Ambos frames coexisten en el call stack. El frame de Student está suspendido mientras Person's `__init__` se ejecuta. `self` en ambos frames apunta al mismo objeto Student instance — `super()` no crea un objeto nuevo, solo redirige la llamada.

Omisión importante: `super()` no retorna la clase padre directamente, retorna un objeto proxy que implementa la MRO. Si hubiera herencia múltiple, `super()` seguiría el orden de la MRO completa, no necesariamente la clase padre inmediata.

Paso 14 de 15 (Student instance completa): La instancia muestra todos los atributos: `age`, `city`, `country`, `firstname`, `gender`, `lastname`, `skills` (apuntando a una lista vacía). `gender` fue asignado por Student's `__init__`, los demás por Person's `__init__` vía `super()`. La lista `skills` aparece como puntero a "empty list" — es un objeto lista independiente en el heap.